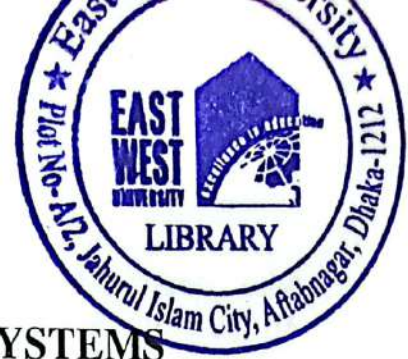




CHAPTER 1

INTRODUCTION TO DIGITAL SYSTEMS

1.1 Introduction

When most of us hear the term *digital*, we immediately think of “digital calculator” or “digital computer”. However, it should be noted that calculators and computers represent only one of the many applications of digital circuits and principles. Digital circuits are used in electronic products such as video games, microwave ovens, and automobile control systems, and in test equipment such as meters, generators, and oscilloscopes. Digital techniques have also replaced a lot of the older “analog circuits” used in consumer products such as radios, TV sets, and high fidelity sound recording and playback equipment.

In this book we are going to study the principles and techniques that are common to all digital systems.

1.2 Digital and Analog Systems

In most of our daily endeavors, we are constantly dealing with *quantities*. Quantities are measured, monitored, recorded, manipulated arithmetically, observed, or in some other way utilized in most physical systems. There are basically two ways of representing the numerical value of quantities: *analog* and *digital*.

In *analog representation* a quantity is represented by a voltage, current, or meter movement that is proportional to the value of that quantity. An example is an automobile speedometer, in which the deflection of the needle is proportional to the speed of the auto. The angular position of the needle represents the value of the auto’s speed, and the needle follows any changes that occur as the auto speeds up or slows down. Analog quantities have an important characteristic: *they can vary over a continuous range of values*. The automobile speed can have any value between zero and, say, 100 mph.

In *digital representation* the quantities are represented not by proportional quantities but by symbols called *digits*. As an example, consider the digital watch, which provides the time of day in the form of decimal digits that represent hours, minutes, and seconds. As we know, the time of day changes continuously, but the digital watch reading does not change continuously; rather it changes in steps of one per second. In other words, this digital representation of the time of day changes in *discrete steps*.

A *digital system* is a combination of devices designed to manipulate physical quantities or information that are represented in digital form; that is, they can take on only discrete values. These devices are most often electronic, but they can be mechanical, magnetic, or pneumatic. Some of the more familiar digital systems include digital computers and calculators, digital audio and video equipment, and the digital telephone system.

An *analog system* contains devices that manipulate physical quantities that are represented in analog form. In an analog system, the quantities can vary over a continuous range of values. Some common analog systems are audio amplifier, magnetic tape recording and playback equipment, and the automobile odometer.

Advantages of Digital Techniques

An increasing majority of applications in electronics, as well as in most other technologies, use digital techniques to perform operations that were once performed using analog methods. The chief reasons for the shift to digital technology are:

1. Digital systems are generally easier to design.
2. Information storage is easy.
3. Accuracy and precision are greater.
4. Operation can be programmed; that is, operation can be controlled by a set of stored instructions.
5. Digital circuits are less affected by noise.
6. More digital circuitry can be fabricated on IC chips.

1.3 Binary Numbers in Digital Systems

In digital systems, information is represented and manipulated in digital form. Most of the digital devices operate in only two stable states and are inherently *binary* in nature. The two stable states of a digital device can be used for representing only two digits. Fortunately, the binary number system has only two digits: 0 and 1, and any number can be represented by using only these two digits. Therefore, *binary number system* is used for representing and manipulating information in digital systems. We shall discuss binary number system in detail in Chapter 2.

In digital systems, binary digits can be represented by any device that has only two operating states or possible conditions. For example, a switch has only two states: open and closed. We can arbitrarily let an open switch represent binary 0 and a closed switch represent binary 1. With this assignment we can now represent any binary number as illustrated in Figure 1.1, where the states of the various switches represent binary number 10010.

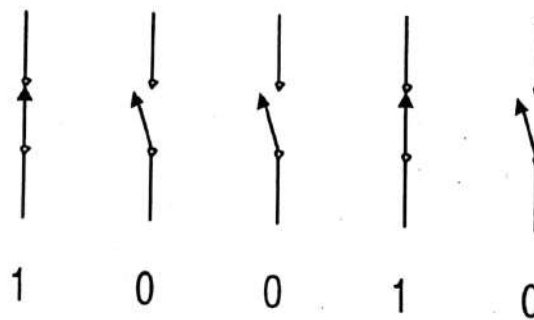


Figure 1.1 Using switches to represent binary numbers.

There are numerous other devices which have only two operating states or can be operated in two extreme conditions, among these are: light bulbs (bright or dark), diode (conducting or nonconducting), relay (energized or deenergized), transistor (cut off or saturated), and magnetic tape (magnetized or demagnetized).

In electronic digital systems, binary digits are represented by voltages that are present at the inputs and outputs of the various circuits. Typically, the binary 0 and binary 1 are represented by two nominal voltage levels. For example, zero volts (0V) might represent binary 0, and +5V might represent binary 1. In actuality, because of circuit variations, the binary 0 and binary 1 would be represented by

voltage ranges. This is illustrated in Figure 1.2(a), where any voltage between 0 and 0.4V represent a binary 0 and any voltage between 2.4 and 5V represent a binary 1. All input and output signals will normally fall within one of these ranges except signal as it sequences through the binary value 10010.

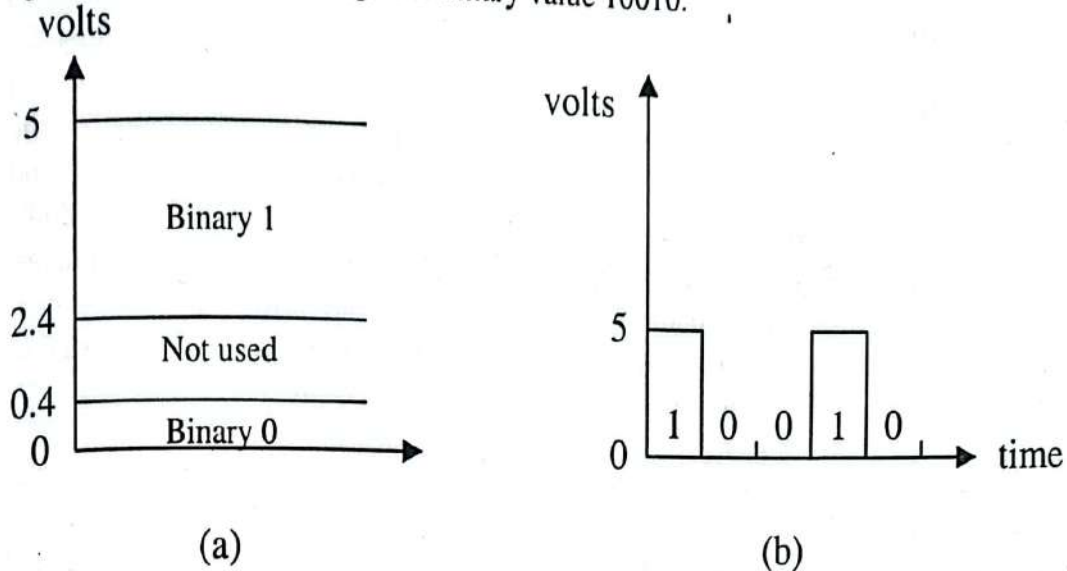


Figure 1.2 (a) Typical voltage assignments in digital system, (b) typical digital signal.

1.4 Binary Logic

Binary logic deals with variables that take on two discrete values and with operations that assume logical meaning. The two values the variables take may be called by different names (e.g., *true* and *false*, *yes* and *no*, etc.), but in the digital system it is convenient to think in terms of binary digits and assign the values of 1 and 0. Binary logic is used to describe, in a mathematical way, the manipulation and processing of binary information. It is particularly suited for the analysis and design of digital systems. The binary logic to be introduced here is equivalent to an algebra called *Boolean algebra*. We shall discuss Boolean algebra in detail in Chapter 3.

1.4.1 Definition of Binary Logic

Binary logic consists of binary variables and logical operations. The variables are designated by letters of the alphabet such as A , B , C , x , y , z , etc., with each variable having two and only two distinct possible values: 0 and 1. There are three basic logical operations: AND, OR, and NOT.

1. **AND:** This operation is represented by a dot or by the absence of an operator. For example, $x \cdot y = z$ or $xy = z$ is read " x AND y is equal to z ". The logical operation AND is interpreted to mean that $z = 1$ if and only if $x = 1$ and $y = 1$; otherwise $z = 0$.
2. **OR:** This operation is represented by a plus sign. For example, $x + y = z$ is read " x OR y is equal to z ", meaning that $z = 1$ if $x = 1$ or $y = 1$ or if both $x = 1$ and $y = 1$. If both $x = 0$ and $y = 0$, then $z = 0$.

3. **NOT:** This operation is represented by prime (sometimes by a bar). For example, $x' = z$ (or $\bar{X} = z$) is read "x not is equal to z", meaning that z is what x is not. In other words, if $x = 1$, then $z = 0$; but if $x = 0$, then $z = 1$.

1.4.2 Truth Table

For each combination of the values of x and y , there is a value of z specified by the definition of the logical operation. These definitions may be listed in a compact form using truth tables. A *truth table* is a table of all possible combinations of the variables showing the relation between the values that the variables may take and the result of the operation. For example, the truth tables for operations AND and OR with variables x and y are obtained by listing all possible values that the variables may have when combined in pairs. The result of the operation for each combination is then listed in a separate row. The truth tables for AND, OR, and NOT are listed in Table 1.1. These tables clearly demonstrate the definition of the operations.

Table 1.1 Truth tables of logical operations.

AND	
$x \ y$	$z = x \cdot y$
0 0	0
0 1	0
1 0	0
1 1	1

OR	
$x \ y$	$z = x + y$
0 0	0
0 1	1
1 0	1
1 1	1

NOT	
x	$z = x'$
0	1
1	0

1.4.3 Switching Circuits

The use of binary variables and the application of binary logic are demonstrated by the simple switching circuits of Figure 1.3. Let the switches A and B represent two binary variables with values equal to 0 when the switch is open and 1 when the switch is closed. Similarly, let the lamp L represent a third binary variable equal to 1 when the light is on and 0 when off. For the switches in series, the light turns on if A and B are closed. For the switches in parallel, the light turns on if A or B is closed. It is obvious that the two circuits can be expressed by means of binary logic with the AND and OR operations, respectively;

$$L = A \cdot B \text{ for the circuit of Figure 1.3(a),}$$

$$L = A + B \text{ for the circuit of Figure 1.3(b).}$$

Electronic digital circuits are sometimes called *switching circuits* because they behave like a switch.

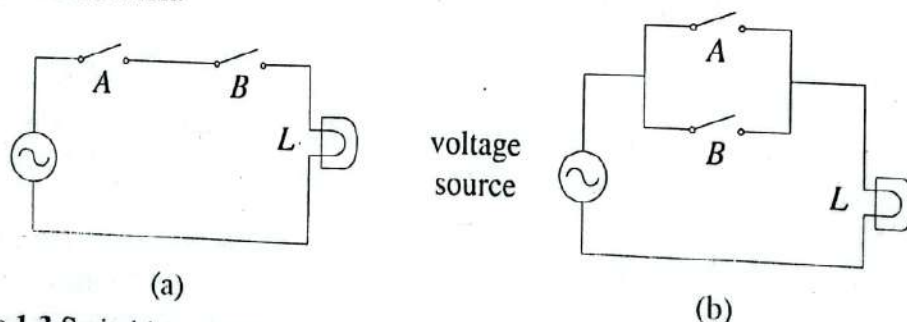


Figure 1.3 Switching circuits that demonstrate binary logic – (a) switches in series – logic AND, (b) switches in parallel – logic OR.

1.4.4 Logic Gates

Electronic digital circuits are also called *logic circuits* because, with the proper input, they establish logical manipulation paths. Logic circuits that perform the logic operations of AND, OR, and NOT are shown with their symbols in Figure 1.4. These circuits, called *logic gates*, are building blocks of digital systems that produce a 0 or 1 output if input logic requirements are satisfied. The NOT gate is sometimes called an *inverter* since it inverts a binary signal.

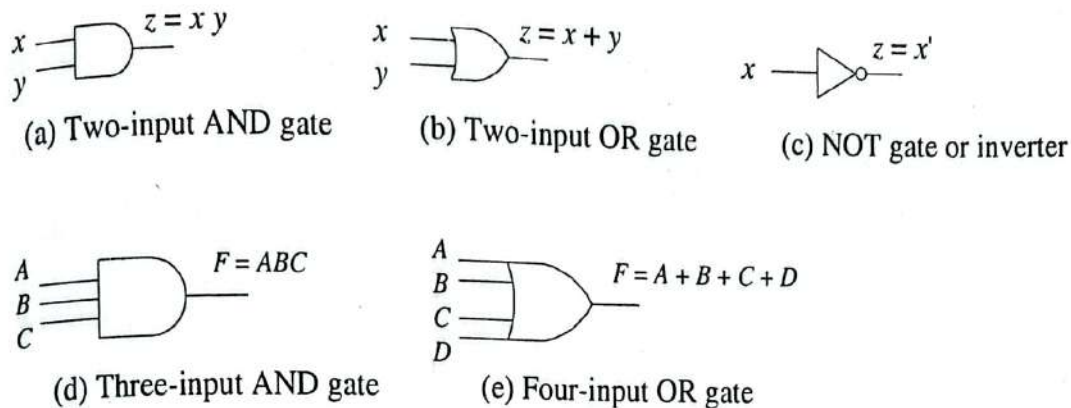


Figure 1.4 Symbols for digital logic gates.

The input signals x and y in the two-input gates of Figure 1.4 may exist in one of four possible states: 00, 01, 10, or 11. These input signals are shown in Figure 1.5, together with the output signals for the AND and OR gates. The timing diagrams in Figure 1.5 illustrate the response of each gate to each of the four possible input binary combinations. The response of NOT gate for input x is also shown in the figure.

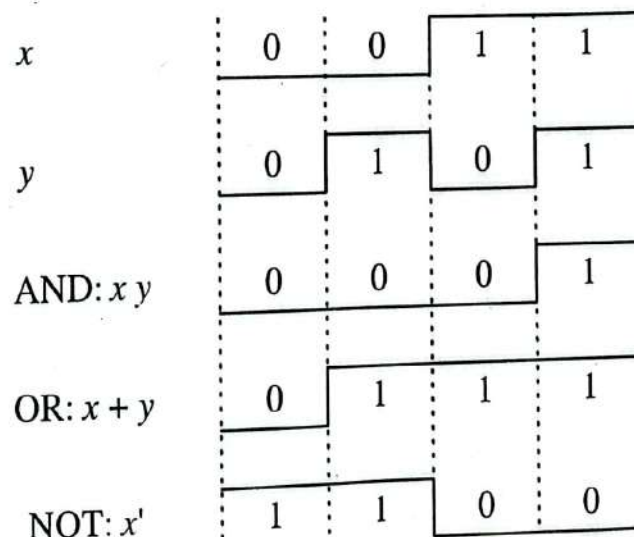


Figure 1.5 Input-output signals for AND, OR, and NOT gates.

AND and OR gates may have more than two inputs. An AND gate with three inputs and an OR gate with four inputs are shown in Figure 1.4. The three-input AND gate responds with an 1 output if all three input signals are 1. The output produces a 0 signal if any input is 0. The four-input OR gate responds with an 1 when any input is

an 1. Its output becomes 0 if all input signals are 0. We shall discuss logic gates in detail in Chapter 4.

1.5 Integrated Circuits

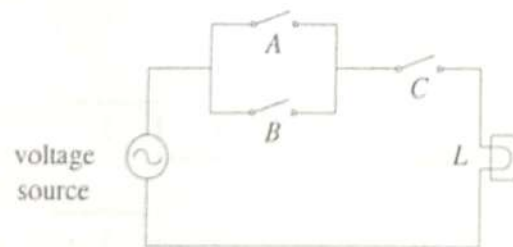
Digital circuits are invariably constructed with integrated circuits. An integrated circuit (abbreviated as IC) is a small silicon semiconductor crystal, called a *chip*, containing electrical components such as transistors, diodes, resistors, and capacitors. The various components are interconnected inside the chip to form an electronic circuit. The chip is mounted on a metal or plastic package, and connections are welded to external pins to form the IC. Digital systems are constructed with ICs.

ICs are classified as follows, depending on the number of gates they contain:

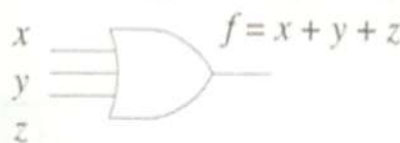
- **Small-scale integration (SSI):** Several logic gates in a single IC.
- **Medium-scale integration (MSI):** IC perform a complete logic function with 10 to 100 gates.
- **Large-scale integration (LSI):** IC performs a complete logic function with more than 100 gates.
- **Very-large-scale integration (VLSI):** IC performs complex logic operation with thousands of gates.

Exercises

- 1.1 Briefly describe the major difference between analog and digital quantities.
- 1.2 What are the advantages of digital techniques over analog?
- 1.3 Express the following switching circuit in binary logic notation.

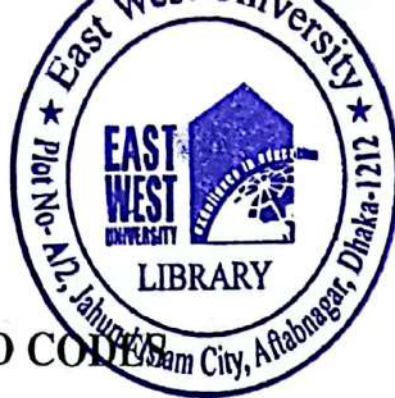


- 1.4 Show the output responses of the following gates using timing diagrams.



Further Reading

1. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.
2. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
3. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.
4. M. Morris Mano, *Digital design*, 3rd ed., Pearson Education, 2002.



CHAPTER 2 NUMBER SYSTEMS AND CODES

2.1 Introduction

We all are familiar with the number system in which an ordered set of ten symbols – 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9, known as *digits* – are used to specify any number. This number system is popularly known as the *decimal number system*. The *radix* or *base* of this number system is 10 (number of distinct digits). Any number is a collection of these digits. For example, 2356.3294 signifies a number with an *integer part* equal to 2356 and a *fractional part* equal to .3294, separated from the integer part with a *radix point* (.), also known as *decimal point*. There are some other systems also used to represent numbers. Some of the other commonly used number systems are *binary*, *octal*, and *hexadecimal* number systems. These number systems are widely used in digital systems and, therefore, the knowledge of these number systems is very essential for understanding, analyzing and designing digital systems.

Digital systems are required to handle data, which may be numerals, alphabets, or special characters. But digital systems use binary signals for their operations. Therefore, the information available in any other form is required to be converted into suitable binary form before it can be processed by digital circuits. This means that the information available in the form of numerals, alphabets, and special characters or in any combination of these must be converted into binary format. To achieve this, a process of coding is employed whereby each numeral, alphabet, or special character is coded in a unique combination of 0s and 1s using a coding scheme, known as a *code*. The process of coding is known as *encoding*.

There can be a variety of coding schemes (codes) to serve different purposes, such as arithmetic operations, data entry, error detection and correction, etc. In digital systems, a large number of codes are in use. Selection of a particular code depends on its suitability for the purpose. In one digital system, different codes may be used for different operations and it may be necessary to convert data from one code to another code.

2.2 Number Systems

In general, in any number system there is an ordered set of symbols known as *digits* with rules defined for performing arithmetic operations like addition, multiplication, etc. A collection of these digits makes a number which in general has two parts – *integer* and *fractional*, set apart by a *radix point* (.), that is

$$(N)_r = \underbrace{d_{n-1}d_{n-2} \cdots d_i \cdots d_1d_0}_{\text{Integer portion}} . \underbrace{d_{-1}d_{-2} \cdots d_{-f} \cdots d_{-m}}_{\text{Fractional portion}}$$

↑
Radix point

where N = a number
 r = radix or base of the number system
 n = number of digits in integer portion

m = number of digits in fractional portion

d_{n-1} = most significant digit (MSD)

d_{-m} = least significant digit (LSD)

and $0 \leq (d_i \text{ or } d_{-j}) \leq (r - 1)$.

To distinguish between numerals of different bases, we enclose the digits of the number in parentheses and write a subscript equal to the base used (except sometimes for decimal numerals, where the context makes it obvious that it is decimal).

The digits in a number are placed side by side and each position in the number is assigned a *weight* of r^i or r^{-j} . Thus, the quantity represented by the number is

$$\begin{aligned}(N)_r &= d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \cdots + d_i r^i + \cdots + d_1 r^1 + d_0 r^0 \\ &\quad + d_{-1} r^{-1} + d_{-2} r^{-2} + \cdots + d_{-j} r^{-j} + \cdots + d_{-m} r^{-m} \\ &= \sum_{i=-m}^{n-1} d_i r^i\end{aligned}$$

For example, the value of the decimal number 2356.3294 is

$$\begin{aligned}(2356.3294)_{10} &= 2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0 + 3 \cdot 10^{-1} + 2 \cdot 10^{-2} + 9 \cdot 10^{-3} + 4 \cdot 10^{-4} \\ &= 2000 + 300 + 50 + 6 + 0.3 + 0.02 + 0.009 + 0.0004 \\ &= 2356.3294\end{aligned}$$

As each position of the number is assigned a weight, this number system is called *positional number system*.

2.3 Complements

Complements are used for simplifying the subtraction operation. There are two types of complements for each base- r system:

- (1) the r 's complement and
- (2) the $(r - 1)$'s complement.

When the value of the base is substituted, the two types receive the names 2's and 1's complement for binary numbers, or 10's and 9's complement for decimal numbers.

2.3.1 The r 's Complement

Given a positive number N in base r with an integer part of n digits, the r 's complement of N is defined as

$$\text{The } r\text{'s complement of } N = \begin{cases} r^n - N & \text{if } N \neq 0 \\ 0 & \text{if } N = 0 \end{cases}$$

The following numerical examples will help clarify the definition

The 10's complement of $52520 = 10^5 - 52520 = 47480$.

[The number of digits in the integer part of the number is $n = 5$.]

The 10's complement of .3267 = $10^0 - 0.3267 = 0.6733$.

[The number of digits in the integer part of the number is $n = 0$.]

The 10's complement of 25.639 = $10^2 - 25.639 = 74.361$.

From the definition and the examples, it is clear that *the 10's complement of a decimal number can be formed by leaving all least significant zeros unchanged, subtracting the first nonzero least significant digit from 10, and then subtracting all other higher significant digits from 9.*

2.3.2 The $(r - 1)$'s Complement

Given a positive number N in base r with an integer part of n digits and a fractional part of m digits, the $(r - 1)$'s complement of N is defined as

The $(r - 1)$'s complement of $N = r^n - r^{-m} - N$.

Some numerical examples follow:

The 9's complement of 52520 = $10^5 - 10^{-0} - 52520 = 100000 - 1 - 52520 = 99999 - 52520 = 47479$. [No fractional part, so $m = 0$.]

The 9's complement of .3267 = $10^0 - 10^{-4} - .3267 = 1 - .0001 - .3267 = .9999 - .3267 = .6732$. [No integer part, so $n = 0$.]

The 9's complement of 25.639 = $10^2 - 10^{-3} - 25.639 = 100 - .001 - 25.639 = 99.999 - 25.639 = 74.360$.

From the examples, we see that *the 9's complement of a decimal number is formed simply by subtracting every digit from 9*. Since the $(r - 1)$'s complement is very easily obtained, it is sometimes convenient to use it when the r 's complement is desired. From the definition and from a comparison of the results obtained in the examples, it follows that *the r 's complement can be obtained from the $(r - 1)$'s complement after the addition of r^{-m}* . For example, the 10's complement of 25.639 is 74.361 and this can be obtained by adding $10^{-3} = 0.001$ to the 9's complement of 25.639, that is 74.360, to give 74.361.

It is worth mentioning that the complement of the complement restores the number to its original value. The r 's complement of N is $r^n - N$ and r 's complement of $(r^n - N)$ is $r^n - (r^n - N) = N$; similarly for the $(r - 1)$'s complement.

2.3.3 Subtraction with r 's Complement

The direct method of subtraction uses the borrow concept. In this method, we borrow a 1 from a higher significant position when the minuend digit is smaller than the corresponding subtrahend digit. This seems to be easiest when people perform subtraction with paper and pencil. When subtraction is implemented by means of digital components, this method is found to be less efficient than the method that uses complements and addition as stated below.

The subtraction of two positive numbers ($M - N$), both of base r , may be done as follows:

1. Find the r 's complement of the subtrahend N and add it to the minuend M .
2. Inspect the result obtained in step 1 for an end carry.
 - (a) If an end carry occurs, the result is a positive number. Discard the end carry to find the magnitude of the result.
 - (b) If an end carry does not occur, the result is a negative number. Take the r 's complement of the number obtained in step 1 to find the magnitude of the result.

The following examples illustrate the procedure:

Example 1:

$$\begin{array}{rcl}
 72532 & \xrightarrow{\quad} & 72532 \\
 (-) 3250 & \xrightarrow{\quad} & (+) 96750 \text{ (10's complement of 3250)} \\
 \hline
 69282 & & (1) 69282 \\
 & \downarrow & \\
 & \text{end carry} &
 \end{array}$$

The result is 69282.

Example 2:

$$\begin{array}{rcl}
 3250 & \xrightarrow{\quad} & 03250 \\
 (-) 72532 & \xrightarrow{\quad} & (+) 27468 \text{ (10's complement of 72532)} \\
 \hline
 - 69282 & & 30718 \text{ (no end carry)}
 \end{array}$$

The result is -69282 (10's complement of 30718)

The proof of the procedure is: The r 's complement of N is $(r^n - N)$. Adding it to M gives $(M + r^n - N)$. For numbers having an integer part of n digits, r^n is equal to a 1 in the $(n+1)$ st position (what has been called the "end carry"). Since both M and N are assumed to be positive, then

$$(a) \quad (M + r^n - N) \geq r^n \text{ if } M \geq N, \text{ or}$$

$$(b) \quad (M + r^n - N) < r^n \text{ if } M < N.$$

In case (a) the result is positive and equal to $M - N$, which is directly obtained by discarding the end carry r^n . In case (b) the result is negative and equal to $-(N - M)$. This case is detected from the absence of an end carry. The result is obtained by taking a second r 's complement and adding a negative sign:

$$-[r^n - (M + r^n - N)] = -(N - M).$$

2.3.4 Subtraction with $(r - 1)$'s Complement

The procedure for subtraction with $(r - 1)$'s complement is exactly the same as the one used with the r 's complement except for one variation, called "end-around carry", as shown below. The subtraction of $M - N$, both positive numbers in base r , may be calculated in the following manner:

1. Find the $(r - 1)$'s complement of the subtrahend N and add it to the minuend M .
2. Inspect the result obtained in step 1 for an end carry.
 - (a) If an end carry occurs, the result is a positive number. Add the end carry to the least significant digit (end-around carry) to find the magnitude of the result.
 - (b) If an end carry does not occur, the result is a negative number. Take the $(r - 1)$'s complement of the number obtained in step 1 to find the magnitude of the result.

The following examples illustrate the procedure:

Example 1:

$$\begin{array}{rcl}
 72532 & \xrightarrow{\quad} & 72532 \\
 (-) 3250 & \xrightarrow{\quad} & (+) 96749 \text{ (9's complement of 03250)} \\
 \hline
 69282 & & (1) 69281 \\
 & & \xrightarrow{+1} \text{ (+) 1 end-around carry} \\
 & & \hline
 & & 69282
 \end{array}$$

The result is 69282.

Example 2:

$$\begin{array}{rcl}
 3250 & \xrightarrow{\quad} & 03250 \\
 (-) 72532 & \xrightarrow{\quad} & (+) 27467 \text{ (9's complement of 72532)} \\
 \hline
 -69282 & & 30717 \text{ (no end carry)}
 \end{array}$$

The result is -69282 (9's complement of 30717).

The proof of the procedure is: The $(r - 1)$'s complement of N is $(r^n - r^{-m} - N)$. Adding it to M gives $(M + r^n - r^{-m} - N)$. For numbers having an integer part of n digits, r^n is equal to a 1 in the $(n + 1)$ st position. Since both M and N are assumed to be positive, then:

- (a) $(M + r^n - r^{-m} - N) \geq r^n - r^{-m}$ if $M \geq N$, or
- (b) $(M + r^n - r^{-m} - N) < r^n - r^{-m}$ if $M < N$.

For numbers having a fractional part of m digits, subtracting r^{-m} is equal to subtracting a 1 (in base r) from the least significant digit ($-m$ th digit). In case (a) the result is positive and equal to $M - N$, which is directly obtained by discarding the end carry r^n and adding a 1 to the least significant digit. In case (b) the result is negative and equal to $-(N - M)$. This case is detected from the absence of an end carry. The result is obtained by taking a second $(r - 1)$'s complement and adding a negative sign:

$$-[r^n - r^{-m} - (M + r^n - r^{-m} - N)] = -(N - M).$$

2.4 Binary Number System

2.4.1 Binary Number Representation

The number system with base (or radix) two is known as the *binary number system*. Only two digits are used to represent numbers in this system and these are 0 and 1. These are known as *bits* (bit is the abbreviation of *binary digit*). It is a positional number system and every position is assigned a weight of 2^i .

Table 2.1 illustrates counting in binary number system. The corresponding decimal numbers are given in the right-hand column. The left-most bit is known as the *most significant bit (MSB)* and the right-most bit is known as the *least significant bit (LSB)*. Any number of 0s can be added to the left of the number without changing the value of the number. In the binary number system, a group of four bits is known as a *nibble*, and a group of eight bits is known as a *byte*.

In binary number system, using n bit, number ranging from 0 to $(2^n - 1)$ can be represented. For example, using 4 bits, numbers ranging from 0 to 15 (that is $2^4 - 1$) can be represented as seen in Table 2.1.

2.4.2 Binary to Decimal Conversion

A binary number can be converted to decimal number by using the formula

$$(N)_2 = \sum_{i=-m}^n d_i 2^i$$

For example,

$$\begin{aligned} (1010.011)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} \\ &= 8 + 0 + 2 + 0 + 0 + 0.25 + 0.125 \\ &= (10.375)_{10} \end{aligned}$$

Table 2.1 4-bit binary numbers and their corresponding decimal numbers.

Binary number	Decimal number
0000	00
0001	01
0010	02
0011	03
0100	04
0101	05
0110	06
0111	07
1000	08
1001	09
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

2.4.3 Decimal to Binary Conversion

Any decimal number can be converted into its equivalent binary number. This conversion is done separately for integer and fractional parts.

Conversion of Integer Part

Let $(N)_{10}$ be a decimal integer number whose binary equivalent is $b_{n-1}b_{n-2} \cdots b_1b_0$, where bs are bits. Therefore, we can write that

$$(N)_{10} = b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \cdots + b_12^1 + b_02^0$$

If we divide $(N)_{10}$ by 2, we get

$$(N/2)_{10} = (b_{n-1}2^{n-2} + b_{n-2}2^{n-3} + \cdots + b_12^0) + b_0$$

where, $(b_{n-1}2^{n-2} + b_{n-2}2^{n-3} + \cdots + b_12^0)$ is the quotient and b_0 is the remainder. In this division, the least significant bit b_0 is separated out. By repeated division of the quotient by 2, we can separate out the higher significant bits. This process is explained below for $(13)_{10}$:

	Quotient	Remainder	
13/2	6	1	LSB
6/2	3	0	
3/2	1	1	
1/2	0	1	MSB

Thus $(13)_{10} = (1101)_2$.

Conversion of Fractional Part

Let $(N)_{10}$ be a decimal fractional number whose binary equivalent is $.b_{-1}b_{-2} \cdots b_{-m}$ and we can write that

$$(N)_{10} = b_{-1}2^{-1} + b_{-2}2^{-2} + \cdots + b_{-m}2^{-m}$$

If we multiply $(N)_{10}$ by 2, we get

$$(N \times 2)_{10} = b_{-1} + (b_{-2}2^{-1} + \cdots + b_{-m}2^{-(m-1)})$$

where, b_{-1} is the integer part and $(b_{-2}2^{-1} + \cdots + b_{-m}2^{-(m-1)})$ is the fraction part of $(N \times 2)_{10}$. In this multiplication, the most significant bit b_{-1} is separated out. By repeated multiplication of the fractional part, we can separate out the lower significant bits. This process is explained below for $(.6875)_{10}$:

$.6875 \times 2 = 1.3750$	$\longrightarrow$	1 MSB
$.3750 \times 2 = 0.7500$	$\longrightarrow$	0
$.7500 \times 2 = 1.5000$	$\longrightarrow$	1
$.5000 \times 2 = 1.0000$	$\longrightarrow$	1 LSB

Thus $(.6875)_{10} = (.1011)_2$.

The conversion of decimal numbers with both integer and fractional parts is done by converting the integer and fraction separately and then combining the two results together. Using the results of the above examples we have

$$(13.6875)_{10} = (1101.1011)_2.$$

2.4.4 2's Complement of Binary Numbers

Given a positive binary number N with an integer part of n bits, the 2's complement of N is defined as

$$\text{The 2's complement of } N = \begin{cases} 2^n - N & \text{if } N \neq 0 \\ 0 & \text{if } N = 0 \end{cases}$$

The following numerical examples will help clarify the definitions.

$$\begin{aligned} \text{The 2's complement of } (101100)_2 &= (2^6)_{10} - (101100)_2 \\ &= (1000000 - 101100)_2 = (010100)_2. \end{aligned}$$

$$\begin{aligned} \text{The 2's complement of } (.0110)_2 &= (2^0)_{10} - (.0110)_2 \\ &= (1 - .0110)_2 = (.1010)_2. \end{aligned}$$

From the definition and the examples, it is clear that *the 2's complement can be formed by leaving all least significant zeros and the first nonzero bit unchanged, and then replacing 1s by 0s and 0s by 1s in all other higher significant bits.*

2.4.5 1's Complement of Binary Numbers

Given a positive binary number N with an integer part of n bits and a fractional part of m bits, the 1's complement of N is defined as

$$\text{The 1's complement of } N = 2^n - 2^{-m} - N.$$

Some numerical examples follow:

$$\begin{aligned} \text{The 1's complement of } (101100)_2 &= (2^6 - 2^0)_{10} - (101100)_2 \\ &= (111111 - 101100)_2 = (010011)_2. \text{ [No fractional part, so } m = 0.] \end{aligned}$$

$$\begin{aligned} \text{The 1's complement of } (.0110)_2 &= (2^0 - 2^{-4})_{10} - (.0110)_2 \\ &= (.1111 - .0110)_2 = (.1001)_2. \end{aligned}$$

From the examples, we see that *the 1's complement of a binary number is formed by changing 1s to 0s and 0s to 1s.* Since the 1's complement is very easily obtained, it is sometimes convenient to use it when the 2's complement is desired. From the definition and from a comparison of the results obtained in the examples, it follows that *the 2's complement can be obtained from the 1's complement after addition of a 1 to the least significant bit.* For example, the 2's complement of $(10110100)_2$ is $(01001100)_2$ and this can be obtained from the 1's complement $(01001011)_2$ by adding 1 to give $(01001100)_2$.

2.5 Binary Arithmetic

We all are familiar with the arithmetic operations such as addition, subtraction, multiplication, and division of decimal numbers. Similar operations can be performed on binary numbers.

2.5.1 Binary Addition

The rules of binary addition are given in Table 2.2.

Table 2.2 Rules of binary addition.

Augend	Addend	Result	Carry	Sum
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	10	1	0

In the first three rows in Table 2.2, the result is of one bit only and there is no *carry*, that is, carry is 0. In the fourth row, the result is of two bits and a *carry* is produced (since in binary two is 10), that is, carry is 1. Similar to decimal addition this carry is added to the next higher bit position. Binary addition is explained using examples below:

Example 1: 1 0 1 1 Augend
 (+) 0 1 0 0 Addend

 1 1 1 1 Sum

Example 2: 1 0 1 1
 (+) 1 1 0 0

 1 0 1 1 1

 └─> Carry

Example 3: (1) (1) ← Carry

 0 1 0 1
 (+) 1 1 0 1

 1 0 0 1 0

 └─> Carry



2.5.2 Binary Subtraction

The rules of binary subtraction are given in Table 2.3.

Table 2.3 Rules of binary subtraction.

Minuend	Subtrahend	Borrow	Difference
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

In Table 2.3, except in the second row, the borrow is 0. We borrow a 1 from a higher significant position when the minuend bit is smaller than the corresponding subtrahend bit. When the borrow is 1, as in the second row, this is to be subtracted from the next higher bit as it is done in decimal subtraction. Binary subtraction is explained using the examples below:

Example 1: 1 0 1 1 Minuend
 (-) 1 0 1 0 Subtrahend

0 0 0 1 Difference

Example 2: (1) ← Borrow
 1 0 1 1
 (-) 0 1 1 0
 0 1 0 1

2.5.3 Binary Multiplication

Binary multiplication is similar to decimal multiplication. In binary, each partial product is either zero (multiplication by 0) or exactly same as the multiplicand (multiplication by 1). An example of binary multiplication is given below:

 1 0 0 1 Multiplicand
 × 1 1 0 1 Multiplier

1 0 0 1
 0 0 0 0
 1 0 0 1
 1 0 0 1

 1 1 1 0 1 0 1 Final product

} Partial Products

In a digital system, the multiplication operation is performed by repeated additions of all partial products to obtain the final product.

2.5.4 Binary Division

Binary division is obtained using the same procedure as decimal division. An example of binary division is given below:

Divisor 1 0 0 1 1 1 0 1 Quotient
 1 1 1 0 1 0 1 Dividend
 1 0 0 1

 1 0 1 1
 1 0 0 1

 1 0 0 1
 1 0 0 1

2.5.5 Signed Binary Numbers

There are three methods of representing signed binary numbers. They are discussed below.

Sign-Magnitude Representation

In the decimal number system a plus (+) sign is used to denote a positive number and a minus (−) sign for denoting a negative number. The plus sign is usually dropped, and the absence of any sign means that the number has positive value. This representation of numbers is known as *signed number*. As is well known, digital systems can understand only two symbols, 0 and 1, therefore, we must use the same symbols to indicate the sign of the number also. Normally, an additional bit is used as the *sign bit* and it is placed as the most significant bit. A 0 is used to represent a positive number and a 1 to represent a negative number. For example, an 8-bit signed number $(01000100)_2$ represents a positive number and its value (magnitude) is $(1000100)_2 = (68)_{10}$. The left most 0 (MSB) indicates that the number is positive. On the other hand, in the signed binary form, $(11000100)_2$ represents a negative number with magnitude $(1000100)_2 = (68)_{10}$. The 1 in the left most position (MSB) indicates that the number is negative and the other seven bits give its magnitude. This kind of representation for signed numbers is known as *signed-magnitude representation*. The user must take care to see the representation used while dealing with the binary numbers.

Table 2.4 gives sign-magnitude numbers represented by 4-bit binary numbers. It can be observed that for an n -bit number, the maximum positive number which can be represented in sign-magnitude representation is $(2^{n-1} - 1)$ and the maximum negative number is $-(2^{n-1} - 1)$. This method has the disadvantage that zero has dual representation (0000 for +0 and 1000 for −0).

1's Complement Representation

In 1's complement representation, a number is made negative by taking its 1's complement. For example, $(+5)_{10}$ is represented by $(0101)_2$ and $(-5)_{10}$ is represented by $(1010)_2$, where $(1010)_2$ is the 1's complement of $(0101)_2$. In this representation, also MSB is 0 for positive numbers and 1 for negative number. In a positive number, all the bits of the number give its magnitude. In a negative number, 1's complement of the bits of the number give its magnitude.

Table 2.4 Sign-magnitude binary number representation using four bits.

Sign-magnitude binary number	Decimal number
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	−0
1001	−1
1010	−2

1011	-3
1100	-4
1101	-5
1110	-6
1111	-7

Table 2.5 gives 1's complement numbers represented by 4-bit binary numbers. It can be observed that for an n -bit number, the maximum positive number which can be represented in 1's complement representation is $(2^{n-1} - 1)$ and the maximum negative number is $-(2^{n-1} - 1)$. This method also has the disadvantage that zero has dual representation (0000 for +0 and 1111 for -0).

2's Complement Representation

In 2's complement representation, a number is made negative by taking its 2's complement. For example, $(+5)_{10}$ is represented by $(0101)_2$ and $(-5)_{10}$ is represented by $(1011)_2$, where $(1011)_2$ is the 2's complement of $(0101)_2$. In this representation also, MSB is 0 for positive numbers and 1 for negative numbers. In a positive number, all the bits of the number give its magnitude. In a negative number, 2's complement of the bits of the number give its magnitude.

Table 2.6 gives 2's complement number representation by 4-bit binary numbers. It can be observed that for an n -bit number, the maximum positive number which can be represented in 2's complement representation is $(2^{n-1} - 1)$ and the maximum negative number is $-(2^{n-1})$. In 2's complement representation, zero has a unique representation (0000). This method is widely used for representing signed numbers.

Table 2.5 1's complement binary number representation using four bits.

1's complement binary number	Decimal number
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-7
1001	-6
1010	-5
1011	-4
1100	-3
1101	-2
1110	-1
1111	-0

Table 2.6 2's complement binary number representation using four bits.

2's complement binary number	Decimal number
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

2.5.6 1's Complement Binary Arithmetic

Binary arithmetic can be easily performed using 1's complement signed binary number representation. But this method is not usually used in digital computers.

Subtraction Using 1's Complement

Binary subtraction can be performed by adding the 1's complement of the subtrahend to the minuend. If an end carry is generated, the answer is positive and add the end carry to the least significant bit (end-around carry) to find the magnitude of the answer. If the end carry is 0, the answer is negative and is in 1's complement form. This method is explained using the examples below:

Example 1:

$$\begin{array}{rcl}
 7 & \rightarrow & 0111 \text{ Minuend} \\
 -5 & \rightarrow & (+) 1010 \text{ 1's complement of subtrahend} \\
 \hline
 +2 & & (1) 0001 \\
 & & \rightarrow (+) 1 \text{ Add end-around carry} \\
 & & \hline
 & & 0010
 \end{array}$$

The answer is 0010, which is equivalent to $(+2)_{10}$.

Example 2:

$$\begin{array}{rcl}
 5 & \rightarrow & 0101 \text{ Minuend} \\
 -7 & \rightarrow & (+) 1000 \text{ 1's complement of subtrahend} \\
 \hline
 -2 & & 1101
 \end{array}$$

The end carry is 0. Therefore, the answer is negative and is in 1's complement form. 1's complement of 1101 is 0010. Therefore, the answer is $(-2)_{10}$.

Addition/Subtraction in 1's Complement Representation

The addition/subtraction of signed binary numbers can be performed using 1's complement representation of both the operands. This method is explained using the examples below:

Example 1: $48 - 23 = 48 + (-23) = +25$
 $48 \rightarrow 00110000$ 8-bit 1's complement representation of +48
 $+ (-23) \rightarrow (+) 11101000$ 8-bit 1's complement representation of -23

$$\begin{array}{r} +25 \\ \hline (1)00011000 \\ \hline \rightarrow (+) 1 \text{ Add end-around carry} \end{array}$$

$$\begin{array}{r} 00011001 \\ \hline \end{array} \text{ 8-bit 1's complement representation of +25}$$

Example 2: $23 - 48 = 23 + (-48) = -25$

$$\begin{array}{r} 23 \rightarrow 00010111 \text{ 8-bit 1's complement representation of +23} \\ + (-48) \rightarrow (+) 11001111 \text{ 8-bit 1's complement representation of -48} \end{array}$$

$$\begin{array}{r} -25 \\ \hline 11100110 \text{ 8-bit 1's complement representation of -25} \end{array}$$

Example 3: $48 - (-23) = 48 + 23 = +71$

$$\begin{array}{r} 48 \rightarrow 00110000 \text{ 8-bit 1's complement representation of +48} \\ + 23 \rightarrow (+) 00010111 \text{ 8-bit 1's complement representation of +23 or -(-23)} \end{array}$$

$$\begin{array}{r} +71 \\ \hline 01000111 \text{ 8-bit 1's complement representation of +71} \end{array}$$

Example 4: $-48 - 23 = (-48) + (-23) = -71$

$$\begin{array}{r} -48 \rightarrow 11001111 \text{ 8-bit 1's complement representation of -48} \\ -23 \rightarrow (+) 11101000 \text{ 8-bit 1's complement representation of -23} \end{array}$$

$$\begin{array}{r} -71 \\ \hline (1)10110111 \\ \hline \rightarrow (+) 1 \text{ Add end-around carry} \end{array}$$

$$\begin{array}{r} 10111000 \\ \hline \end{array} \text{ 8-bit 1's complement representation of -71}$$

From the above examples, we observe the following:

1. If the two operands are of the opposite sign, the result is to be obtained by the rule of subtraction using 1's complement.
2. If the two operands are of the same sign, the sign bit of the result (MSB) is to be compared with the sign bit of the operands. If the sign bits are same, the result is correct and is in 1's complement form. If the sign bits are not same, there is a problem of *overflow*, i.e., the result can not be accommodated using eight bits and the result is to be interpreted suitably. The result in this case will consist of nine bits, i.e., carry and eight bits, and the carry bit will give the sign of the number.

2.5.7 2's Complement Binary Arithmetic

Binary arithmetic can be easily performed using 2's complement signed binary number representation. This method is usually used in digital computers. In this arithmetic, the problem of subtraction is changed to that of addition. Therefore, it is possible to use the circuits designed for binary addition to perform the binary subtraction also. This concept eliminates the need of additional circuits for subtraction, rather the same adder circuit are used for both the operations. This makes design of arithmetic circuits very convenient and cheaper (see Chapter 7).

Subtraction Using 2's Complement

Binary subtraction can be performed by adding the 2's complement of the subtrahend to the minuend. If an end carry is generated, the answer is positive and discard the end carry to find the answer. If the end carry is 0, the answer is negative and is in 2's complement form. This method is explained using the examples below:

Example 1:

$$\begin{array}{rcl}
 7 & \rightarrow & 0111 \text{ Minuend} \\
 -5 & \rightarrow & (+) 1011 \text{ 2's complement of subtrahend} \\
 \hline
 +2 & & (1)0010 \\
 & \searrow & \rightarrow \text{Discard end carry}
 \end{array}$$

The answer is 0010, which is equivalent to $(+2)_{10}$.

Example 2:

$$\begin{array}{rcl}
 5 & \rightarrow & 0101 \text{ Minuend} \\
 -7 & \rightarrow & (+) 1001 \text{ 2's complement of subtrahend} \\
 \hline
 -2 & & 1110
 \end{array}$$

The end carry is 0. Therefore, the answer is negative and is in 2's complement form. 2's complement of 1110 is 0010. Therefore, the answer is $(-2)_{10}$.

Addition/Subtraction in 2's Complement Representation

The addition/subtraction of signed binary numbers can be conveniently performed using 2's complement representation of both the operands. This is the method most commonly used when these operations are performed in digital computers. This method is explained using the examples below:

Example 1: $48 - 23 = 48 + (-23) = +25$

$$\begin{array}{rcl}
 48 & \rightarrow & 00110000 \text{ 8-bit 2's complement representation of } +48 \\
 +(-23) & \rightarrow & (+) 11101001 \text{ 8-bit 2's complement representation of } -23 \\
 \hline
 +25 & & (1)00011001 \text{ 8-bit 2's complement representation of } +25 \\
 & \searrow & \rightarrow \text{Discard end carry}
 \end{array}$$

Example 2: $23 - 48 = 23 + (-48) = -25$

$$\begin{array}{rcl}
 23 & \rightarrow & 00010111 \text{ 8-bit 2's complement representation of } +23 \\
 + (-48) & \rightarrow & (+) 11010000 \text{ 8-bit 2's complement representation of } -48 \\
 \hline
 -25 & & 11100111 \text{ 8-bit 2's complement representation of } -25
 \end{array}$$

Example 3: $48 - (-23) = 48 + 23 = +71$

$$\begin{array}{rcl}
 48 & \rightarrow & 00110000 \text{ 8-bit 2's complement representation of } +48 \\
 + 23 & \rightarrow & (+) 00010111 \text{ 8-bit 2's complement representation of } +23 \text{ or } -(-23) \\
 \hline
 +71 & & 01000111 \text{ 8-bit 2's complement representation of } +71
 \end{array}$$

Example 4: $-48 - 23 = (-48) + (-23) = -71$

$$\begin{array}{rcl}
 -48 & \rightarrow & 11010000 \text{ 8-bit 2's complement representation of } -48 \\
 -23 & \rightarrow & (+) 11101001 \text{ 8-bit 2's complement representation of } -23 \\
 \hline
 -71 & & (1)10111001 \text{ 8-bit 2's complement representation of } -71 \\
 & \rightarrow & \text{Discard end carry}
 \end{array}$$

From the above examples, we observe the following:

1. If the two operands are of the opposite sign, the result is to be obtained by the rule of subtraction using 2's complement.
2. If the two operands are of the same sign, the sign bit of the result (MSB) is to be compared with the sign bit of the operands. If the sign bits are same, the result is correct and is in 2's complement form. If the sign bits are not same, there is a problem of *overflow*, i.e., the result can not be accommodated using eight bits and the result is to be interpreted suitably. The result in this case will consist of nine bits, i.e., carry and eight bits, and the carry bit will give the sign of the number.

2.6 Octal Number System

2.6.1 Octal Number Representation

The number system with base (or radix) eight is known as the *octal number system*. In this system, eight digits, 0, 1, 2, 3, 4, 5, 6, and 7 are used to represent numbers. Similar to decimal and binary number systems, it is also a positional number system and every position is assigned a weight of 8^i . For example, $(6327.4051)_8$ is an octal number, which is equivalent to $(3287.5100098)_{10}$.

2.6.2 Octal to Decimal Conversion

An octal number can be converted to decimal number by using the formula

$$(N)_8 = \sum_{i=-m}^n d_i 8^i$$

For example,

$$\begin{aligned} (6327.4051)_{10} &= 6 \times 8^3 + 3 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} \\ &\quad + 0 \times 8^{-2} + 5 \times 8^{-3} + 1 \times 8^{-4} \\ &= 3072 + 192 + 16 + 7 + 4/8 + 0 + 5/512 + 1/4096 \\ &= (3287.5100098)_{10} \end{aligned}$$

2.6.3 Decimal to Octal Conversion

The conversion from decimal to octal is similar to the conversion procedure for decimal to binary conversion. The only difference is that number 8 is used in place of 2 for division in the case of integer part and for multiplication in the case of fractional part. The process is explained below for $(3287.5100098)_{10}$:

Integer part

	Quotient	Remainder
3287/8	410	7 LSD
410/8	51	2
51/8	6	3
6/8	0	6 MSD

Thus $(3287)_{10} = (6327)_8$.

Fractional part

$$\begin{aligned} .5100098 \times 8 &= 4.0800784 \rightarrow 4 \text{ MSD} \\ .0800784 \times 8 &= 0.6406272 \rightarrow 0 \\ .6406272 \times 8 &= 5.1250176 \rightarrow 5 \\ .1250176 \times 8 &= 1.0001408 \rightarrow 1 \text{ LSD (up to four places after the} \\ &\quad \text{radix point)} \end{aligned}$$

Thus $(.5100098)_{10} = (.4051)_8$.

Therefore, $(3287.5100098)_{10} = (6327.4051)_8$

From the above example, we observe that the conversion for fractional part may not be exact. In general, an approximate equivalent can be determined by terminating the process of multiplication by eight at the desired places after the radix point.

2.6.4 Octal to Binary Conversion

Octal numbers can be converted into equivalent binary numbers by replacing each octal digit by its 3-bit equivalent binary. Table 2.7 gives octal digits and their binary equivalents.

Table 2.7 Binary and decimal equivalents of octal digits

Octal	Decimal	Binary
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111

The procedure is illustrated for $(736.42)_8$. From Table 2.7, we observe the binary equivalent of 7, 3, 6, 4, and 2 as 111, 011, 110, 100, and 010, respectively. Therefore,

$$(736.42)_8 = (111\ 011\ 110\ .\ 100\ 010)_2 = (111011110.100010)_2$$

2.6.5 Binary to Octal Conversion

Binary numbers can be converted into equivalent octal numbers by making groups of three bits starting from LSB and moving towards MSB for integer part of the number and then replacing each group of three bits by its octal representation. For fractional part, the groupings of three bits are used starting from the binary point. The procedure is illustrated below:

- (a) $11001110001.000101111001 = 011\ 001\ 110\ 001\ .\ 000\ 101\ 111\ 001 = (3161.0571)_8$
- (b) $1011011110.11001010011 = 001\ 011\ 011\ 110\ .\ 110\ 010\ 100\ 110 = (1336.6246)_8$
- (c) $111110001.10011001101 = 111\ 110\ 001\ .\ 100\ 110\ 011\ 010 = (761.4632)_8$

From the above examples we observe that in forming the 3-bit grouping, 0s may be required to complete the first (MSD) group in the integer part and the last (LSD) group in the fractional part.

2.6.6 Decimal to Binary Conversion via Octal and Vice Versa

Large decimal numbers can be easily converted to equivalent binary numbers by converting the decimal number to equivalent octal numbers and then to binary numbers. Similarly, large binary numbers can be converted to equivalent decimal numbers by converting the binary numbers to equivalent octal numbers and then to decimal numbers. The procedure is illustrated using the following examples.

Example 1: $(1325.65625)_{10}$ **Integer part**

	Quotient	Remainder
1325/8	165	5 LSD
165/8	20	5
20/8	2	4
2/8	0	2 MSD

Thus, $(1325)_{10} = (2455)_8$.**Fractional part**

$$\begin{aligned} .65625 \times 8 &= 5.25000 \longrightarrow 5 \text{ MSD} \\ .25000 \times 8 &= 2.00000 \longrightarrow 2 \text{ LSD} \end{aligned}$$

Thus, $(.65625)_{10} = (.52)_8$.

$$\begin{aligned} \text{Therefore, } (1325.65625)_{10} &= (2455.52)_8 = (010 \ 100 \ 101 \ 101 . 101 \ 010)_2 \\ &= (010100101101.101010)_2 \end{aligned}$$

Example 2: $(010100101101.101010)_2 = (010 \ 100 \ 101 \ 101 . 101 \ 010)_2$

$$\begin{aligned} &= (2455.52)_8 \\ &= 2 \times 8^3 + 4 \times 8^2 + 5 \times 8^1 + 5 \times 8^0 + 5 \times 8^{-1} + 2 \times 8^{-2} \\ &= 1024 + 256 + 40 + 5 + 5/8 + 2/64 \\ &= (1325.65625)_{10} \end{aligned}$$

2.6.7 Octal Arithmetic

Octal arithmetic rules are similar to the decimal or binary arithmetic. Normally, we are not interested in performing octal arithmetic operations using octal representation of numbers. This number system is normally used to enter long strings of binary data into a digital computer. This makes the task of entering binary data into a computer easier. Arithmetic operations can be performed by converting the octal numbers to binary numbers and then using the rules of binary arithmetic. The procedure is illustrated using the following examples.

Example 1:

$$\begin{array}{rcl} (23)_8 & \longrightarrow & 010 \ 011 \\ + (67)_8 & \longrightarrow & (+) 110 \ 111 \\ \hline (112)_8 & \longrightarrow & 1 \ 001 \ 010 \end{array}$$

Example 2:

$$\begin{array}{rcl} (53)_8 & \longrightarrow & 101 \ 011 \longrightarrow 101 \ 011 \\ - (37)_8 & \longrightarrow & (-) 011 \ 111 \longrightarrow (+) 100 \ 001 \text{ 2's complement of } (37)_8 \\ \hline (14)_8 & \longrightarrow & 1 \ 001 \ 100 \\ & & \downarrow \\ & & \text{Discard end carry} \end{array}$$

Example 3:

$$\begin{array}{rcl}
 (26)_8 & \longrightarrow & 010\ 110 \longrightarrow 010\ 110 \\
 -(75)_8 & \longrightarrow & (-) 111\ 101 \longrightarrow (+) 000\ 011 \text{ 2's complement of } (75)_8 \\
 \hline
 -(47)_8 & \longrightarrow & 011\ 001 \text{ 2's complement of } (47)_8, \\
 & & \text{that is } -(47)_8
 \end{array}$$

Multiplication and division can also be performed using the binary representation of octal numbers and then making use of multiplication and division rules of binary numbers.

2.6.8 Application of Octal Number System

In digital systems, binary numbers are required to be entered and certain results or status signals are required to be displayed. It is highly inconvenient to handle long strings of binary numbers. It may cause errors also. Therefore, octal numbers are used for entering the binary data and displaying certain information. Therefore, the knowledge of octal number system is very important for the efficient use of digital systems. For example, the binary number 011111110 can easily be remembered as 376 and can be entered as 376 using octal keys. Since digital circuits can process only 0s and 1s, the octal numbers have to be converted into binary form using special circuit before being processed by the digital circuits.

2.7 Hexadecimal Number System**2.7.1 Hexadecimal Number Representation**

Hexadecimal number system is very popular in computer uses. The base for hexadecimal number system is 16. In this system, 16 digits are required to represent the numbers. These digits are numerals 0 through 9 and alphabets A through F. Table 2.8 gives hexadecimal digits with their decimal and binary equivalents. Similar to decimal, binary, and octal number systems, it is also a positional number system and every position is assigned a weight of 16^i . For example, $(3A.2F)_{16}$ is a hexadecimal (hex) number that is equivalent to $(58.1836)_{10}$.

2.7.2 Hexadecimal to Decimal Conversion

A hexadecimal number can be converted to decimal number by using the formula

$$(N)_{16} = \sum_{i=-m}^n d_i 16^i$$

For example,

$$\begin{aligned}
 (3A.2F)_{16} &= 3 \times 16^1 + 10 \times 16^0 + 2 \times 16^{-1} + 15 \times 16^{-2} \\
 &= 48 + 10 + 2/16 + 15/156 = (58.1836)_{10}.
 \end{aligned}$$

The fractional part may not be an exact equivalent and therefore, may give a small error.

Table 2.8 Decimal and binary equivalents of hexadecimal digits

Hexadecimal digits	Decimal numbers	Binary numbers
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

2.7.3 Decimal to Hexadecimal Conversion

For conversion from decimal to hexadecimal, the procedure used in binary and octal systems is applicable, using 16 as the dividing (for integer part) and multiplying (for fractional part) factor. The procedure is illustrated for $(675.625)_{10}$.

Integer part

	Quotient	Remainder (Hex)
675/16	42	3 LSD
42/16	2	A
2/16	0	2 MSD

Thus, $(675)_{10} = (2A3)_{16}$

Fractional part

$$625 \times 16 = 10.00 \rightarrow A$$

Thus, $(.625)_{10} = (A)_{16}$

Therefore, $(675.625)_{10} = (2A3.A)_{16}$

2.7.4 Hexadecimal to Binary Conversion

Hexadecimal numbers can be converted into equivalent binary numbers by replacing each hexadecimal digit by its equivalent 4-bit binary number. The procedure is illustrated for $(2F9A.5B)_{16}$. From Table 2.8, find the binary equivalent for each hexadecimal digit.

$$\begin{aligned}
 (2F9A.5B)_{16} \\
 &= (0010 \ 1111 \ 1001 \ 1010 \ 0101 \ 1011)_2 \\
 &= (001011111001101001011011)_2
 \end{aligned}$$

2.7.5 Binary to Hexadecimal Conversion

Binary numbers can be converted into the equivalent hexadecimal numbers by making groups of four bits starting from LSB and moving towards MSB for integer part and then replacing each group of four bits by its hexadecimal representation. For the fractional part, the above procedure is repeated starting from the bit next to the binary point and moving towards the right. The procedure is illustrated for

$$\begin{aligned} & (10100110101111.00011110101101)_2 \\ & (10100110101111.00011110101101)_2 \\ & = (0010\ 1001\ 1010\ 1111.0001\ 1110\ 1011\ 0100)_2 \\ & = (29AF.1EB4)_{16} \end{aligned}$$

From the above example, we observe that in forming the 4-bit grouping 0s may be required to complete the first (MSD) group in the integer part and the last (LSD) group in the fractional part.

2.7.6 Hexadecimal to Octal Conversion and Vice Versa

Hexadecimal numbers can be converted to equivalent octal numbers by converting the hexadecimal number to equivalent binary number and then to octal number. Similarly, octal numbers can be converted to equivalent hexadecimal numbers by converting the octal number to equivalent binary number and then to hexadecimal number. The process is illustrated using the following examples.

Example 1: $(A72E.BF85)_{16}$

$$\begin{aligned} & = (1010\ 0111\ 0010\ 1110.1011\ 1111\ 1000\ 0101)_2 \\ & = (001\ 010\ 011\ 100\ 101\ 110.101\ 111\ 111\ 000\ 010\ 100)_2 \\ & = (123456.577024)_8 \end{aligned}$$

Example 2: $(247.36)_8$

$$\begin{aligned} & = (010\ 100\ 111.011\ 110)_2 \\ & = (0\ 1010\ 0111.0111\ 1000)_2 \\ & = (A7.78)_{10} \end{aligned}$$

2.7.7 Decimal to Binary Conversion via Hexadecimal and Vice Versa

Large decimal numbers can be easily converted to equivalent binary numbers by converting the decimal number to equivalent hexadecimal number and then to binary number. Similarly, large binary numbers can be converted to equivalent decimal numbers by converting the binary number to equivalent hexadecimal number and then to decimal number. The procedure is illustrated using the following examples.

Example 1: $(1325.65625)_{10}$

Integer part

	Quotient	Remainder (Hex)
1325/16	82	D LSD
82/16	5	2
5/16	0	5 MSD

Thus, $(1325)_{10} = (52D)_{16}$.

Fractional part

$$.65625 \times 16 = 10.50000 \rightarrow A \text{ MSD}$$

$$.50000 \times 16 = 8.00000 \rightarrow 8 \text{ LSD}$$

$$\text{Thus, } (.65625)_{10} = (.A8)_{16}$$

$$\text{Therefore, } (1325.65625)_{10} = (52D.A8)_{16}$$

$$= (0101 \ 0010 \ 1101 \ . \ 1010 \ 1000)_2$$

$$= (010100101101.10101000)_2$$

Example 2:

$$(010100101101.10101000)_2$$

$$= (0101 \ 0010 \ 1101 \ . \ 1010 \ 1000)_2$$

$$= (52D.A8)_{16}$$

$$= 5 \times 16^2 + 2 \times 16^1 + 13 \times 16^0 + 10 \times 16^{-1} + 8 \times 16^{-2}$$

$$= 1280 + 32 + 13 + 10/16 + 8/256$$

$$= (1325.65625)_{10}$$

2.7.8 Hexadecimal Arithmetic

The rules for arithmetic operations with hexadecimal numbers are similar to the rules for decimal, binary, and octal systems. Normally, we are not interested in performing hexadecimal arithmetic operations using hexadecimal representation of numbers. Similar to octal system, this number system is also used to enter long strings of binary data into a digital computer. Arithmetic operations can be performed by converting the hexadecimal numbers to binary numbers and then using the rule of binary arithmetic. The procedure is illustrated using the following examples.

Example 1:

$$\begin{array}{rcl} (7F)_{16} & \rightarrow & 0111 \ 1111 \\ + (BA)_{16} & \rightarrow & (+) 1011 \ 1010 \\ \hline (139)_{16} & \rightarrow & 1 \ 0011 \ 1001 \end{array}$$

Example 2:

$$\begin{array}{rcl} (3F)_{16} & \rightarrow & 0011 \ 1111 \rightarrow 0011 \ 1111 \\ - (5C)_{16} & \rightarrow & -0101 \ 1100 \rightarrow + 1010 \ 0100 \text{ 2's complement of } (5C)_{16} \\ \hline - (1D)_{16} & \rightarrow & 1110 \ 0011 \text{ 2's complement of } (1D)_{16}, \\ & & \text{that is } -(1D)_{16} \end{array}$$

Example 3:

$$\begin{array}{rcl} (C0)_{16} & \rightarrow & 1100 \ 0000 \rightarrow 1100 \ 0000 \\ - (7A)_{16} & \rightarrow & -0111 \ 1010 \rightarrow + 1000 \ 0110 \text{ 2's complement of } (7A)_{16} \\ \hline (46)_{16} & \rightarrow & 1 \ 0100 \ 0110 \\ & & \downarrow \text{Discard end carry} \end{array}$$

Multiplication and division can also be performed using the binary representation of hexadecimal numbers and then making use of multiplication and division rules of binary numbers.

2.7.9 Application of Hexadecimal Number System

Like octal numbers, hexadecimal numbers are also used for entering the binary data and displaying certain information in digital computers. In practice, hexadecimal numbers are more frequently used than octal numbers for this purpose. Therefore, the knowledge of hexadecimal number system is very important for the efficient use of digital systems. For example, the binary number 11111110 can easily be remembered as FE and can be entered as FE using hexadecimal keys.

2.8 Binary Codes

Digital systems represent and manipulate not only binary numbers but also many other discrete elements of information such as numeric, alphabets and special characters. As digital circuits process data in the binary format, these discrete elements of information are represented in digital systems by various binary codes. Although, in every code used the information is represented in binary form, the interpretation of this binary information is possible only if the code in which this information is available is known. For example, the binary number 1000001 represents 65 (decimal), 41 (decimal) in BCD and alphabet A in ASCII code. A user must be very careful about the code being used while interpreting information available in the binary format. Binary codes are also used for error detection and error correction in digital systems.

To represent a group of 2^n distinct elements in a binary code requires a minimum of n bits. This is because it is possible to arrange n bits in 2^n distinct ways. For example, a group of four distinct quantities can be represented by a two-bit code, with each quantity assigned to one of the following bit combinations: 00, 01, 10, 11. A group of eight elements requires a three-bit code, with each element assigned to one and only one of the following: 000, 001, 010, 011, 100, 101, 110, 111. The examples show that the distinct bit combinations of an n -bit code can be found by counting in binary from 0 to $(2^n - 1)$. Some bit combinations are unassigned when the number of elements of the group to be coded is not a multiple of the power of 2. The ten decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 are an example of such a group. A binary code that distinguishes among ten elements must contain at least four bits; three bits can distinguish a maximum of eight elements. Four bits can form 16 distinct combinations, but since only ten digits are coded, the remaining six combinations are unassigned and not used.

Some of the commonly used binary codes are discussed here.

2.8.1 BCD Code

In BCD (binary-coded decimal) code, decimal digits 0 through 9 are represented (coded) by their binary equivalents using four bits as shown in Table 2.9. Each

decimal digit of a decimal number is represented by this four-bit code individually. For example, $(23)_{10}$ is represented by 0010 0011 using BCD code, rather than $(10111)_2$. From this it is observed that it requires more number of bits to code a decimal number using BCD code than using the straight binary form. However, in spite of this disadvantage it is very convenient and useful code for input and output operations in digital systems. BCD code is also known as 8421 code; 8, 4, 2, and 1 are the weight of the four bits of the binary code of each decimal digit similar to straight binary number system. Therefore, this is a *weighted* code and arithmetic operations can be performed using this code, which will be discussed in Chapter 7. BCD code is the most commonly encountered decimal code.

2.8.2 Excess-3 Code

In excess-3 code, decimal digit is coded into a four-bit binary code, the code for each decimal digit is obtained by adding decimal 3 to the BCD code of the digit as shown in Table 2.9. For example, decimal digit 2 is coded as $0010 + 0011 = 0101$ in Excess-3 code. It is not a weighted code. Like BCD code, each decimal digit of a decimal number is represented by the four-bit Excess-3 code individually. For example, $(23)_{10}$ is represented by 0101 0110 using Excess-3 code. Excess-3 code is a self-complementing code, which means 1's complement of the coded number yields 9's complement of the number itself. For example, Excess-3 code of decimal number 23 is 01010110. Its 1's complement is 10101001, which is Excess-3 code for decimal number 76, which is 9's complement of decimal number 23. The self-complementing property of this code helps considerably in performing subtraction operation in digital systems.

Table 2.9 BCD and Excess-3 codes for decimal digits 0 through 9.

Decimal digit	BCD code $B_3 B_2 B_1 B_0$	Excess-3 code $E_3 E_2 E_1 E_0$
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

2.8.3 Gray Code

Gray code is a very useful code in which a decimal number is represented in binary form in such a way so that each Gray-code number differs from the preceding and

the succeeding number by a single bit. For example, the Gray code for decimal number 5 is 0111 and for 6 is 0101. These two codes differ by only one bit position (third from the left). This code is used extensively for shaft encoders because of this property. It is not a weighted code.

The Gray code is a reflected code and can be constructed using the rules given below:

- (i) A one-bit Gray code has two code words 0 and 1 representing decimal numbers 0 and 1 respectively.
- (ii) (a) An n -bit ($n \geq 2$) Gray code will have first 2^{n-1} Gray codes of $(n-1)$ -bits written in order with a leading 0 appended.
 (b) The last 2^{n-1} Gray codes will be equal to the Gray code words of an $(n-1)$ -bit Gray code, written in reverse order (assuming a mirror placed between first 2^{n-1} and last 2^{n-1} Gray codes) with a leading 1 appended.

The procedure is illustrated below:

1-bit Gray code is constructed using rule (i) as below:

Decimal number	Gray code
0	0
1	1

2-bit Gray code is constructed using rule (ii) and Gray code of 1-bit as below:

Decimal number	Gray code
0	00
1	01
2	11
3	10

3-bit gray code is constructed using 2-bit Gray code as below:

Decimal number	Gray code
0	000
1	001
2	011
3	010
4	110
5	111
6	101
7	100

4-bit gray code is constructed using 3-bit Gray code as below:

Decimal number	Gray code
0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

We can also construct the Gray code of a decimal number directly from its binary representation as follows:

Let $b_{n-1}b_{n-2} \cdots b_1b_0$ be the n -bit binary representation of the given decimal number and $g_{n-1}g_{n-2} \cdots g_1g_0$ be the equivalent n -bit Gray code. Then the Gray code can be constructed using the equation

$$g_i = \begin{cases} b_i & \text{if } i = n-1 \\ b_i \oplus b_{i+1} & \text{otherwise} \end{cases}$$

where $\oplus$ indicates modulo-2 sum, according to the following rules

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

The procedure is illustrated using the following examples.

Example 1: The given decimal number is 10 and its binary equivalent is $b_3b_2b_1b_0 = 1010$. Then

$$g_3 = b_3 = 1$$

$$g_2 = b_2 \oplus b_3 = 0 \oplus 1 = 1$$

$$g_1 = b_1 \oplus b_2 = 1 \oplus 0 = 1$$

$$g_0 = b_0 \oplus b_1 = 0 \oplus 1 = 1$$

and the equivalent Gray code is $g_3g_2g_1g_0 = 1111$.

Example 2: The given decimal number is 14 and its binary equivalent is $b_3b_2b_1b_0 = 1110$. Then

$$g_3 = b_3 = 1$$

$$g_2 = b_2 \oplus b_3 = 1 \oplus 1 = 0$$

$$g_1 = b_1 \oplus b_2 = 1 \oplus 1 = 0$$

$$g_0 = b_0 \oplus b_1 = 0 \oplus 1 = 1$$

and the equivalent Gray code is $g_3g_2g_1g_0 = 1001$.

We can also reconstruct the binary representation of a decimal number from its Gray code:

Let $g_{n-1}g_{n-2} \cdots g_1g_0$ be the given n -bit Gray code and $b_{n-1}b_{n-2} \cdots b_1b_0$ be the n -bit binary representation of its equivalent decimal number. Then the binary representation can be constructed using the equation

$$b_i = \begin{cases} g_i & \text{if } i = n-1 \\ g_i \oplus b_{i+1} & \text{otherwise} \end{cases}$$

The procedure is illustrated using the following examples.

Example 1: The given Gray code is $g_3g_2g_1g_0 = 1111$ and its equivalent binary representation is $b_3b_2b_1b_0$. Then

$$b_3 = g_3 = 1$$

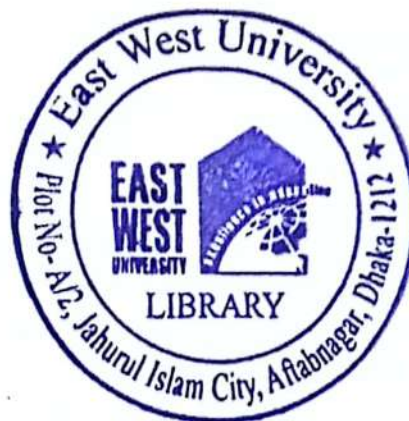
$$b_2 = g_2 \oplus b_3 = 1 \oplus 1 = 0$$

$$b_1 = g_1 \oplus b_2 = 1 \oplus 0 = 1$$

$$b_0 = g_0 \oplus b_1 = 1 \oplus 1 = 0$$

and the binary representation is $b_3b_2b_1b_0 = 1010$, which is equivalent to decimal number 10.

Example 2: The given Gray code is $g_3g_2g_1g_0 = 1001$ and its equivalent binary representation is $b_3b_2b_1b_0$. Then



$$b_3 = g_3 = 1$$

$$b_2 = g_2 \oplus b_3 = 0 \oplus 1 = 1$$

$$b_1 = g_1 \oplus b_2 = 0 \oplus 1 = 1$$

$$b_0 = g_0 \oplus b_1 = 1 \oplus 1 = 0$$

and the binary representation is $b_3b_2b_1b_0 = 1110$, which is equivalent to decimal number 14.

2.8.4 Error-Detection Code

Parity Bit

Binary information can be transmitted from one place to another by any communication medium. Any external noise introduced into the communication medium may change some of the bits from 0 to 1 or vice versa. The purpose of an error-detection code is to detect such bit-reversal errors. One of the most common ways to achieve error detection is by means of a *parity bit*. A parity bit is an extra bit included with a message to make the total number of 1s transmitted either odd (*odd parity*) or even (*even parity*). A message of four bits and a parity bit P are shown in Table 2.10. If an odd parity is adopted, the P bit is chosen such that the total number of 1s is odd in the five bits that constitute the message and P . If an even parity is adopted, the P bit is chosen so that the total number of 1s in the five bits is even. In a particular situation, one or the other parity is adopted, with even parity being more common.

Table 2.10 Parity bit.

Odd parity		Even parity	
Message	P	Message	P
0000	1	0000	0
0001	0	0001	1
0010	0	0010	1
0011	1	0011	0
0100	0	0100	1
0101	1	0101	0
0110	1	0110	0
0111	0	0111	1
1000	0	1000	1
1001	1	1001	0
1010	1	1010	0
1011	0	1011	1
1100	1	1100	0
1101	0	1101	1
1110	0	1110	1
1111	1	1111	0

The parity bit is helpful in detecting errors during the transmission of information from one place to another. This is done in the following manner. An even parity is generated in the sending end for each message transmission. The message, together with the parity bit, is transmitted to its destination. The parity of the received data is checked in the receiving end. If the parity of the received information is not even, it means that at least one bit has changed value during the transmission. This method detects one, three, or any odd combination of errors in each message that is transmitted. An even combination of errors is undetected. Additional error-detection schemes may be needed to take care of an even combination of errors. If an error is detected, the data is retransmitted.

Cyclic Redundancy Check (CRC)

One of the most common and popular error-detecting codes is the *Cyclic Redundancy Check (CRC)*. In this code, an n -bit sequence F , called *Frame Check Sequence (FCS)*, is appended with a k -bit message M to form the $(k + n)$ -bit transmitted message T . The FCS F is determined in such a way that the transmitted message T is exactly divisible by an $(n + 1)$ -bit pattern P . The receiver then divides the received message by the pattern P and if there is no remainder, the received message is assumed to be error-free.

In CRC process all arithmetic is done as modulo-2 arithmetic. Modulo-2 arithmetic uses binary arithmetic with no carries, which is shown in Table 2.11. The modulo-2 arithmetic is illustrated using the following example.

Example: The modulo-2 addition, subtraction, and multiplications are shown below:

$$\begin{array}{r}
 1111 \\
 + 1010 \\
 \hline
 0101
 \end{array}
 \qquad
 \begin{array}{r}
 1110 \\
 - 0101 \\
 \hline
 1011
 \end{array}
 \qquad
 \begin{array}{r}
 11001 \\
 \times 11 \\
 \hline
 11001 \\
 11001 \\
 \hline
 101011
 \end{array}$$

Table 2.11 Modulo-2 arithmetic.

$a \ b$	$a + b$	$a - b$	$a \times b$
0 0	0	0	0
0 1	1	1	1
1 0	1	1	1
1 1	0	0	1

Now, we see how the n -bit FCS is determined from k -bit message M and $(n + 1)$ -bit pattern P to form the $(k + n)$ -bit transmitted message T . The $(k + n)$ -bit transmitted message T is formed by concatenating k -bit message M and n -bit FCS F .

T can be formed by shifting M n -bits left with padded out the vacant positions by 0 and then adding F with the shifted M . This operation can be mathematically expressed as

$$T = 2^n M + F$$

This is true, because shifting a number n -bit left is equivalent to multiplying the number by 2^n . The CRC process requires that T/P has no remainder. Suppose that we divided $2^n M$ by P

$$\frac{2^n M}{P} = Q + \frac{R}{P}$$

There is a quotient Q and a remainder R . Because division is modulo-2, the remainder R is always at least one bit shorter than the divisor P . That means, if P is an $(n + 1)$ -bit pattern, the R will be a number of at most n -bit. We will use this remainder R as our FCS F . Thus

$$T = 2^n M + R$$

Now divide T by P :

$$\frac{T}{P} = \frac{2^n M + R}{P} = \frac{2^n M}{P} + \frac{R}{P} = Q + \frac{R}{P} + \frac{R}{P} = Q$$

This is true, as any binary number added to itself modulo-2 yields zero. There is no remainder, and, therefore, T is exactly divisible by P . Thus FCS is easily generated by dividing $2^n M$ by P and using the remainder as the FCS. On reception the transmitted message T is divided by P and if there is no remainder, then the message is error-free, otherwise the transmitted message has error. The CRC process is illustrated using the following example.

Example: Message $M = 1010001101$ (10 bits) and Pattern $P = 110101$ (6 bits). The FCS will be a 5-bit number.

$2^5 M = 101000110100000$. Dividing $2^5 M$ by P we get

$$\begin{array}{r}
 1101010110 \\
 P \rightarrow 110101 \overline{) 101000110100000} \leftarrow 2^n M \\
 \underline{110101} \\
 111011 \\
 \underline{110101} \\
 111010 \\
 \underline{110101} \\
 111110 \\
 \underline{110101} \\
 101100 \\
 \underline{110101} \\
 110010 \\
 \underline{110101} \\
 01110 \leftarrow R
 \end{array}$$

The remainder is added to $2^n M$ to give

$$\begin{array}{r}
 2^n M \longrightarrow 101000110100000 \\
 R \longrightarrow + 01110 \\
 \hline
 T \longrightarrow 101000110101110
 \end{array}$$

which is transmitted. If there is no error, the receiver receives T intact. The received message is divided by P :

$$\begin{array}{r}
 1101010110 \leftarrow Q \\
 P \rightarrow 110101 \overline{) 101000110101110} \leftarrow T \\
 \underline{110101} \\
 111011 \\
 \underline{110101} \\
 111010 \\
 \underline{110101} \\
 111110 \\
 \underline{110101} \\
 101111 \\
 \underline{110101} \\
 110101 \\
 \underline{110101} \\
 0 \leftarrow R
 \end{array}$$

Because there is no remainder, it is assumed that there have been no error.

The pattern P is chosen to be one bit larger than the desired FCS, and the exact bit pattern chosen depends on the type of errors expected. At minimum, both the MSB and LSB of P must be 1. Four versions of P are widely used:

CRC-12: 1100000001111

CRC-16: 11000000000000101

CRC-CCIT: 10001000000100001

CRC-32: 100000100110000010001110110110111

20.8.5 ASCII Character Code

Many applications of digital systems require the handling of data not only of numbers, but also of letters. For instance, an insurance company with thousands of policyholders will use a computer to process its files. To represent the names and other pertinent information, it is necessary to formulate a binary code for the letters of the alphabet. In addition, the same binary code must represent numerals and special characters such as \$. An alphanumeric character set is a set of elements that include the 10 decimal digits, the 26 letters of the alphabet, and a number of special characters.

The standard binary code for the alphanumeric characters is ASCII (American Standard Code for Information Interchange). It uses seven bits to code 128 characters, as shown in Table 2.12. The seven bits of the code are designated by b_6 through b_0 , with b_6 being the MSB. The letter A, for example, is represented in ASCII as 1000001 (column 100, row 0001). The ASCII code contains 94 graphic characters that can be printed and 34 nonprinting characters used for various control functions. The graphic characters consists of the 26 upper case letters (A through Z), the 26 lower case letters (a through z), the 10 numerals (0 through 9), and 32 special printable characters such as %, *, and \$.

Table 2.12 American Standard Code for Information Interchange.

	$b_6b_5b_4$							
$b_3b_2b_1b_0$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	:	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	-	o	DEL

ASCII is a seven-bit code, but most computers manipulate an eight-bit quantity as a single unit called *byte*. Therefore, ASCII characters most often are stored one per byte. The extra bit is sometimes used for other purposes, depending on the application. For example, some printers recognize eight-bit ASCII characters with the MSB set to 0. Additional 128 eight-bit characters with the MSB set to 1 are used for other symbols such as the Greek alphabet or italic type font. When used in data communication, the MSB may be employed to indicate the parity of the character.

Exercise

- 2.1 List the first 16 numbers in base 12. Use the letters A and B to represent the last two digits.
- 2.2 What is the largest binary number that can be obtained with 16 bits? What is its decimal equivalent?
- 2.3 Convert the following binary numbers to decimal equivalent.
 - (a) 10101 (b) 1011.101 (c) 101101.1101 (d) 111001
 - (e) 101001 (f) 11111110 (g) 1100100 (h) 1101.0011
 - (i) 1010.1010 (j) 0.111000 (k) 101110 (l) 1110101.11
 - (m) 110110100
- 2.4 Convert the following decimal numbers to binary, octal, and hexadecimal equivalents.
 - (a) 367 (b) 235.8125 (c) 10.0101 (d) 37
 - (e) 255 (f) 15 (g) 26.25 (h) 11.75 (i) 0.1
 - (j) 1231 (k) 673.23 (l) 10^4 (m) 1998
 - (n) 7562.45 (o) 1938.257 (p) 175.175
- 2.5 Convert the following numbers from decimal to octal and then to binary.
 - (a) 375 (b) 249 (c) 27.125
- 2.6 Convert the following binary numbers to octal and then to decimal.
 - (a) 11011100.101010 (b) 01010011.010101
 - (c) 10110011 (d) 11010111
- 2.7 Convert the following numbers from decimal to hexadecimal and then to binary.
 - (a) 375 (b) 249 (c) 27.125
- 2.8 Convert the following binary numbers to hexadecimal and then to decimal.
 - (a) 11011100.101010 (b) 01010011.010101 (c) 10110011
- 2.9 Convert the following binary numbers to octal and hexadecimal numbers.
 - (a) 1010111 (b) 111101000.0111 (c) 11101
- 2.10 Convert the following octal numbers to decimal, binary, and hexadecimal.
 - (a) 623 (b) 443
- 2.11 Convert the following hexadecimal numbers to decimal, binary, and octal.
 - (a) F3A7C2 (b) 2AC5

- 2.12** Convert the following numbers with the indicated bases to decimal.
 (a) $(12121)_3$ (b) $(4310)_5$ (c) $(50)_7$ (d) $(198)_{12}$
- 2.13** The $(r - 1)$'s complement of base-6 numbers is called the 5's complement.
 (a) Determine a procedure for obtaining the 5's complement of base-6 numbers.
 (b) Obtain the 5's complement of $(543210)_6$.
- 2.14** Perform the following binary arithmetic operations.
 (a) $1011.01 + 0101.10 + 1000.01$ (b) $11011.101 - 1101.011$
 (c) 11.01×10.11 (d) $1011 + 1101$
 (e) $1010.1101 + 101.01$ (f) 110110×110101
 (g) $11111111 \div 101$
- 2.15** Perform the following arithmetic operations in octal number system.
 (a) $237 + 542$ (b) 145×24 (c) $426 - 247$
- 2.16** Perform the following arithmetic operations in hexadecimal number system.
 (a) $23A + 5C2$ (b) $14B \times 24$ (c) $42A - 2F7$
- 2.17** Find the 9's complement of the following 8-digit decimal numbers.
 (a) 12349876 (b) 00980100 (c) 90009951 (d) 00000000
- 2.18** Find the 10's complement of the following 6-digit decimal numbers.
 (a) 123900 (b) 090657 (c) 100000 (d) 000000
- 2.19** Write the 10's and 2's complement of the following decimal numbers.
 (a) 3578 (b) 9090 (c) 136.8
- 2.20** Write the 2's and 1's complement of the following binary numbers.
 (a) 1101.01 (b) 1010101 (c) 00001
 (d) 10101110 (e) 10000001 (f) 10000000
 (g) 00000001 (h) 00000000
- 2.21** Perform subtraction with the following unsigned decimal numbers by taking both the 10's and 9's complement of the subtrahend.
 (a) $5250 - 1321$ (b) $1753 - 8640$
 (c) $20 - 100$ (d) $1200 - 250$
- 2.22** Perform the following subtractions using ordinary, 1's complement, and 2's complement methods.
 (a) $1010 - 01101$ (b) $0010 - 1010$
 (c) $01000 - 01001$ (d) $01100 - 00011$
 (e) $0011.1001 - 0001.1110$ (f) $11010 - 10000$
 (g) $11010 - 1101$ (h) $100 - 110000$
 (i) $1010100 - 1010100$

- 2.23** The binary numbers listed have a sign in the leftmost position and, if negative, are in 2's complement form. Perform the arithmetic operations indicated.
- (a) $101011 + 111000$ (b) $001110 + 110010$ (c) $111001 - 001010$ (d) $101011 - 100110$
- 2.24** Perform the following arithmetic operations in binary using both the signed-1's-complement representation and signed-2's-complement representation for negative numbers.
- (a) $(+42) + (-13)$ (b) $(-42) - (-13)$
- 2.25** Determine the value of base x if $(211)_x = (152)_8$.
- 2.26** Noting that $3^2 = 9$, formulate a simple procedure for converting base-3 numbers directly to base-9. Use the procedure to convert $(2110201102220112)_3$ to base-9.
- 2.27** Write the following decimal numbers in BCD and Excess-3 codes.
- (a) 4793 (b) 46 (c) 327.89 (d) 20.305
 (e) 13597 (f) 93286 (g) 99880 (h) 8620
 (i) 2421
- 2.28** Encode the following decimal numbers in Gray code.
- (a) 46 (b) 30 (c) 55 (d) 21
- 2.29** Decode the following Gray codes to decimal numbers.
- (a) 1010111 (b) 0011001 (c) 1111111 (d) 0000000 (e) 0000001
- 2.30** If the Gray code for a word is $g_{n-1}g_{n-2} \cdots g_i \cdots g_1g_0$, prove that the i th bit of the corresponding binary number $b_{n-1}b_{n-2} \cdots b_i \cdots b_1b_0$ is given by
- $$b_i = \begin{cases} g_i & \text{if } i = n-1 \\ g_i \oplus g_{i+1} \oplus \cdots \oplus g_{n-2} \oplus g_{n-1} & \text{otherwise} \end{cases}$$
- 2.31** List the ten BCD digits with both odd and even parity in the leftmost position.
- 2.32** Attach an even parity bit as MSB for ASCII code.
- 2.33** Attach an odd parity bit as MSB for ASCII code.
- 2.34** Encode the following texts in ASCII using 8-bit code with the leftmost bit always 0.
- (a) Dhanmondi, Dhaka - 1205.
 (b) $4 + 5 - 2$
 (c) Digital Logic Design
- 2.35** Write your full name in ASCII using 8-bit code with the leftmost bit always 0. Include a space between names and a period after a middle initial.
- 2.36** Decode the following ASCII code:
- 1001010 1101111 1101000 1101110 0100000 1000100 1101111 1100101

- 2.37 Show the bit configuration that represents the decimal number 295 (a) in binary, (b) in BCD, (c) in Excess-3, and (d) in ASCII
- 2.38 How many printing characters are there in ASCII? How many of them are not letters or numerals?
- 2.39 The state of a 12-bit register is 010110010111. What is its content if it represents: (a) three decimal digits in BCD; (b) three decimal digits in the Excess-3 code?
- 2.40 Find the number of bits required to encode:
(a) 56 elements of information and (b) 130 elements of information
- 2.41 For $P = 110011$ and $M = 11100011$, find the CRC code.
- 2.42 For $P = 11001$ and $M = 10011011100$, find the CRC code.
- 2.43 How many bits of memory are required for storing 100 names of a group of people, assuming that no name occupies more than 20 characters (including spaces)? Assume 7-bit ASCII code with parity bit.
- 2.44 A line printer is capable of printing 132 characters in a single line and each character is represented by ASCII code. How many bits are required to print each line?

Further Reading

1. Zvi Kohavi, *Switching and Finite Automata Theory*, 2nd ed., McGraw-Hill Higher Education, 1990.
2. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.
3. R. P. Jain, *Modern Digital Electronics*, 2nd ed., Tata McGraw-Hill Publishing Company Limited, India, 1998.
4. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
5. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
6. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
7. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.
8. M. Morris Mano, *Digital Design*, 3rd ed., Pearson Education, 2002.